Логический тип данных bool

В реальной жизни перед нами очень часто возникают вопросы, на которые можно ответить однозначно: «Да» или «Нет». Яблоко является фруктом? Да! Вам нравится сладкое? Нет!

Рассмотрим утверждение: «Яблоко — это фрукт». Является ли это правдой? Да! Яблоко действительно является фруктом. Или как насчет: «Я люблю сладкое». Абсолютная ложь (тьфу!).

Подобные **стейтменты**, которые имеют только два возможных исхода — да/правда или нет/ложь, настолько распространены, что многие языки программирования добавили специальный тип для работы с ними — логический тип данных. В языке C++ это **логический тип данных bool** (от англ. **«boolean»**).

Оглавление:

1. Переменные логического типа данных
2. Использование логического типа данных в ветвлениях if
3. Возвращаемые значения логического типа данных
4. Тест

Переменные логического типа данных

Логические переменные — это переменные, диапазон которых состоит только из двух возможных значений: `true` (`1`) и `false` (`0`).

Для объявления логической переменной используется **ключевое слово bool**:

```
bool b;
```

Инициализировать логическую переменную или выполнить операцию присваивания можно с помощью **ключевых слов** `true` или `false`:

```
bool b1 = true; // инициализация значением
bool b2{false}; // прямая инициализация
bool b3 {true}; // uniform-инициализация (C++11)
b3 = false; // операция присваивания
```

Аналогично работе унарного оператора минус (`-`), с помощью которого мы можем сделать число отрицательным, с помощью логического оператора НЕ (`!`) мы можем изменить `true` на `false` и наоборот (`false` на `true`):

```
bool b1 = true; // b1 = true
bool b2{false}; // значение b2 - true
```

На самом деле, логические значения не сохраняются как `true` или `false`. Они обрабатываются в виде целых чисел: вместо `true` — единица, вместо `false` — ноль.

Следовательно, если мы попытаемся вывести логические значения с помощью `std::cout`, то увидим либо `0`, либо `1`:

```
#include <iostream>

int main()
{
    std::cout << true << std::endl; // вместо true напишется
    std::cout << !true << std::endl; // вместо !true ноль

    bool b(false);
    std::cout << b << std::endl; // b - false (0)
    std::cout << !b << std::endl; // !b - true (1)
    return 0;
}
```

Результат выполнения программы:

```
1
0
0
1
```

Если вы хотите, чтобы std::cout выводил `true` или `false` (вместо целых чисел), то тогда используйте манипулятор форматирования **std::boolalpha**:

```
#include <iostream>

int main()
{
    std::cout << true << std::endl;
    std::cout << false << std::endl;

    std::cout << std::boolalpha; // выводим логическим значением как "true" или "false"

    std::cout << true << std::endl;
    std::cout << false << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
1
0
true
false
```

Использование логического типа данных в ветвлениях if

Очень часто логические переменные используются в ветвлениях if. Ветвление if выглядит следующим образом:

```
if (выражение) стейтмент1;
```

Либо:

```
if (выражение) стейтмент1;
else стейтмент2;
```

`(выражение)` ещё называется **«условием»**, либо **«условным выражением»**.

В обоих случаях, если результатом условия является ненулевое значение, то выполняется `стейтмент1`. Если же результатом условия является нулевое значение, то выполняется `стейтмент2`.

Помните, что `true` — это `1` (ненулевое значение), а `false` — это `0` (нулевое значение).

Теперь рассмотрим пример в коде:

```
if (true) // true - ненулевое значение
    std::cout << "The condition is true!" << std::endl;
else
    std::cout << "The condition is false!" << std::endl;
```

Результат:

```
The condition is true!
```

Что здесь делается? Во-первых, мы начинаем с условия if, которым является логическое значение `true`, т.е. `1` (ненулевое значение), что означает, что выполняться будет `стейтмент1`.

Следующая программа работает аналогично:

```
bool b(false);
if (b)
    std::cout << "b is true!" << std::endl;
else
    std::cout << "b is false!" << std::endl;
```

Результат:

```
b is false!
```

Здесь, при проверке условия, переменная `b` имеет значение `false`, `false` — это `0`. Следовательно, первый стейтмент под if (который `true`) пропускается, а второй, который под `else` (`false`) — выполняется.

А теперь рассмотрим пример посложнее. Оператор равенства (`==`) используется для сравнения двух чисел (являются ли они равными). Оператор `==` возвращает `true`, если операнды равны и `false`, если таковыми не являются:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;

    if (x == 0)
        std::cout << "The value is zero" << std::endl;
    else
        std::cout << "The value is non-zero" << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
Enter an integer: 4
The value is non-zero
```

Давайте разберемся, что и как здесь работает. Во-первых, мы просим пользователя ввести целое число. После этого, с помощью оператора `==`, мы проверяем, является ли пользовательское число нулевым. В вышеприведенном примере `4` не равно `0`, поэтому оператор `==` определяет условие как `false`. Следовательно, выполняется `стейтмент2` (тот, который под `else`), где мы выводим `The value is non-zero`.

Возвращаемые значения логического типа данных

Логические значения часто используются в качестве **возвращаемых значений** в функциях. Названия таких функций очень часто начинаются со слов `is` (например, `isEqual`) или `has` (например, `hasCommonDivisor`).

Рассмотрим следующий пример:

```
#include <iostream>

// Возвращает true, если x и y равны, в противном случае - возвращает false
bool isEqual(int x, int y)
{
    return (x == y); // оператор == возвращает true, если x равно y, в противном случае - false
}

int main()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y;
    std::cin >> y;

    if (isEqual(x, y))
        std::cout << x << " and " << y << " are equal" << std::endl;
    else
        std::cout << x << " and " << y << " are not equal" << std::endl;

    return 0;
}
```

Результат выполнения программы:

```
Enter an integer: 5
Enter another integer: 5
5 and 5 are equal
```

Как это работает? Во-первых, мы указываем значения переменным `x` и `y`. Затем проверяется условие, что приводит к вызову функции isEqual(5, 5). Внутри этой функции наши два числа сравниваются между собой (`5 == 5`), что приводит к возврату значения `true` (так как `5 = 5`). Значение `true` возвращается обратно в caller. Так как условие истинно, то выполняется `стейтмент1`, который выводит `5 and 5 are equal`.

К логическим значениям нужно немного привыкнуть, но как только вы это сделаете, то сами удивитесь, насколько они удобны и просты.

Во всех примерах, приведенных выше, в наших условиях были либо логические значения (`true` или `false`), либо логические переменные, либо функции, которые возвращают логическое значение. А что произойдет, если мы не будем использовать логическое значение в условиях? Правильно! Если результатом условия будет любое ненулевое значение, то выполняться будет `стейтмент1`.

Поэтому, если попробовать сделать что-то вроде следующего:

```
if (x)
    std::cout << "hi";
else
    std::cout << "bye";
```

То результатом будет `hi`, так как `4` является ненулевым значением.

Тест

Что такое простое число? Правильно! Это целое положительное число больше единицы, которое делится без остатка либо на себя, либо на единицу. Напишите программу, которая просит пользователя ввести простое целое число, меньшее 10. Если пользователь ввел одно из следующих чисел: 2, 3, 5 или 7 — программа должна вывести `The digit is prime`, в противном случае — `The digit is not prime`.

> ***Подсказка:*** Используйте ветвление if для сравнения чисел и логические значения для отслеживания того, является ли пользовательское число простым или нет.

Ответ

```
#include <iostream>

bool isPrime(int x)
{
    if (x == 2)
        return true;
    if (x == 3)
        return true;
    if (x == 5)
        return true;
    if (x == 7)
        return true;
    return false;
}

int main()
{
    std::cout << "Enter a single digit integer: ";
    int x;
    std::cin >> x;

    bool prime = isPrime(x);

    if (prime)
        std::cout << "The digit is prime" << std::endl;
    else
        std::cout << "The digit is not prime" << std::endl;

    return 0;
}
```